

Natural Language Processing

Text Preprocessing, Bag-of-Words and TF-IDF

Topics Covered

1. Tokenization
2. Stopword Removal
3. Stemming
4. Lemmatization
5. Bag-of-Words (BoW) Model
6. Term Frequency – Inverse Document Frequency (TF-IDF)

1. Introduction to Text Preprocessing

Natural Language Processing (NLP) enables computers to understand, interpret, and generate human language. Raw text collected from documents, social media, or the web is unstructured and noisy — it contains punctuation, mixed case, irrelevant words, and different grammatical forms of the same word. Machine learning algorithms cannot work directly on raw strings; they require clean, structured, numeric input. Text preprocessing is the pipeline of steps that converts raw, unstructured text into a clean and standardized form suitable for further analysis, feature extraction, and modelling.

A typical NLP pipeline consists of the following stages, each of which is covered in this file with both theoretical explanation and a hands-on Python implementation:

- Tokenization — breaking text into smaller units (words/sentences).
- Stopword Removal — eliminating common words that add little semantic value.
- Stemming — reducing words to their root form using crude rules.
- Lemmatization — reducing words to their dictionary (base) form using vocabulary and grammar.
- Feature Extraction — converting cleaned text into numeric vectors using Bag-of-Words and TF-IDF, so that it can be fed into machine learning models.

Why Preprocessing is Necessary

- Reduces vocabulary size, which reduces computational cost and sparsity.
- Removes noise (punctuation, HTML tags, special characters) that carries no meaning.
- Normalizes different forms of a word (e.g., 'running', 'ran', 'runs' → 'run') so the model treats them as the same concept.
- Improves the accuracy of downstream tasks such as classification, clustering, sentiment analysis, and information retrieval.

2. Environment Setup

Before starting the practicals, install the required Python libraries. NLTK is used for tokenization, stopwords, stemming and lemmatization; scikit-learn is used for Bag-of-Words and TF-IDF vectorization.

```
pip install nltk scikit-learn
```

NLTK requires certain resource packages (tokenizer models, stopword lists, WordNet corpus) to be downloaded once before use:

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('averaged_perceptron_tagger')
```

3. Tokenization

3.1 Theory

Tokenization is the process of splitting a stream of text into smaller, meaningful units called tokens. A token can be a word, a number, a punctuation mark, or a sentence, depending on the level of tokenization used. It is usually the very first step of any NLP pipeline because every later step (stopword removal, stemming, lemmatization, vectorization) operates on tokens rather than on the raw string.

Types of Tokenization

- Word Tokenization — splits text into individual words and punctuation marks. Example: "NLP is fun!" → ['NLP', 'is', 'fun', '!']
- Sentence Tokenization — splits a paragraph into individual sentences using punctuation and language rules (handles abbreviations like 'Dr.', 'e.g.' correctly).
- Sub-word / Whitespace Tokenization — splits purely on whitespace, or into sub-word units (used in deep learning models like BERT).

Challenges in Tokenization

- Handling contractions such as "don't" → "do" + "n't".
- Distinguishing sentence-ending periods from periods in abbreviations ("Mr.", "U.S.A.").
- Handling hyphenated words, URLs, emails, and hashtags.

3.2 Practical Implementation

The following program demonstrates word tokenization and sentence tokenization using NLTK.

```
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize

text = "Natural Language Processing is amazing! It helps computers "\
       "understand human language. Dr. Smith uses NLP for research."

# Sentence Tokenization
sentences = sent_tokenize(text)
print("Sentence Tokens:")
for i, s in enumerate(sentences, 1):
    print(i, '-', s)

# Word Tokenization
words = word_tokenize(text)
print("\nWord Tokens:")
print(words)
print("\nTotal Words:", len(words))
```

Output:

```
Sentence Tokens:
1 - Natural Language Processing is amazing!
2 - It helps computers understand human language.
```

3 - Dr. Smith uses NLP for research.

Word Tokens:

```
['Natural', 'Language', 'Processing', 'is', 'amazing', '!', 'It', 'helps',  
'computers', 'understand', 'human', 'language', '.', 'Dr.', 'Smith',  
'uses', 'NLP', 'for', 'research', '.']
```

Total Words: 20

Observation: notice that "Dr." was correctly kept as a single token instead of being split at the period, showing NLTK's abbreviation handling, while sentence tokenization correctly identified 3 sentences despite the abbreviation.

4. Stopword Removal

4.1 Theory

Stopwords are extremely common words in a language (such as 'is', 'the', 'a', 'and', 'of', 'in') that occur frequently but carry little or no distinctive meaning for tasks like text classification, search, or topic modelling. Removing them reduces the size of the vocabulary, decreases the dimensionality of the resulting feature vectors, and helps the model focus on the words that actually differentiate one document from another.

When to Remove Stopwords

- Useful for: Bag-of-Words, TF-IDF, topic modelling, search/information retrieval, text classification.
- Usually NOT removed for: machine translation, sentiment analysis where negations matter (e.g., "not"), text generation, and modern transformer-based models which learn context including stopwords.

Note: Removing the word 'not' can invert the sentiment of a sentence. Always check whether stopwords removal helps or hurts your specific task before applying it blindly.

4.2 Practical Implementation

NLTK provides a built-in list of stopwords for many languages. The example below removes English stopwords from the tokenized text.

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

text = "This is a simple example to show how stopwords removal works in NLP"
tokens = word_tokenize(text)

stop_words = set(stopwords.words('english'))
filtered_tokens = [w for w in tokens if w.lower() not in stop_words]

print("Original Tokens:", tokens)
print("Number of Stopwords in English:", len(stop_words))
print("Filtered Tokens:", filtered_tokens)
```

Output:

Original Tokens: ['This', 'is', 'a', 'simple', 'example', 'to', 'show', 'how', 'stopword', 'removal', 'works', 'in', 'NLP']

Number of Stopwords in English: 179

Filtered Tokens: ['simple', 'example', 'show', 'stopword', 'removal', 'works', 'NLP']

5. Stemming

5.1 Theory

Stemming is the process of reducing a word to its root or base form by chopping off suffixes and prefixes, using fixed rules — without necessarily considering whether the result is a valid dictionary word. For example, 'playing', 'played', and 'plays' would all be reduced to 'play'. Because stemming uses simple heuristic rules rather than true linguistic knowledge, it is fast but can sometimes produce non-words (called 'over-stemming'), e.g., 'university' → 'univers'.

Common Stemming Algorithms

- Porter Stemmer — the oldest and most widely used algorithm; applies a series of 5 phases of rule-based suffix stripping. Moderate aggressiveness.
- Snowball Stemmer (Porter2) — an improved, more accurate and more consistent version of the Porter algorithm; supports multiple languages.
- Lancaster Stemmer — a more aggressive stemmer that produces shorter stems; can over-stem in many cases.

5.2 Practical Implementation

The example below compares the Porter Stemmer, Snowball Stemmer, and Lancaster Stemmer on the same set of words.

```
from nltk.stem import PorterStemmer, SnowballStemmer, LancasterStemmer

words = ['playing', 'played', 'plays', 'studies', 'studying',
         'university', 'universal', 'connection', 'connected']

porter = PorterStemmer()
snowball = SnowballStemmer('english')
lancaster = LancasterStemmer()

print(f"{'Word':<14}{'Porter':<14}{'Snowball':<14}{'Lancaster':<14}")
for w in words:

    print(f"w:<14}{porter.stem(w):<14}{snowball.stem(w):<14}{lancaster.stem(w):<14}")
```

Output:

Word	Porter	Snowball	Lancaster
playing	play	play	play
played	play	play	play
plays	play	play	play
studies	studi	studi	study
studying	studi	studi	study
university	univers	univers	univers
universal	univers	univers	univers
connection	connect	connect	connect
connected	connect	connect	connect

6. Lemmatization

6.1 Theory

Lemmatization also reduces a word to its base form, called the 'lemma', but unlike stemming it uses a vocabulary and morphological analysis of words, usually with the help of a dictionary such as WordNet. It considers the context and part of speech (POS) of a word to return a valid dictionary word. For example, 'better' is lemmatized to 'good' (with the correct POS tag), and 'studies' is lemmatized to 'study' — always producing real words, unlike stemming.

Stemming vs Lemmatization

Aspect	Stemming	Lemmatization
Approach	Rule-based suffix stripping	Dictionary + morphological analysis
Output	May not be a real word	Always a valid dictionary word
Speed	Faster	Slower (needs POS info)
Accuracy	Lower	Higher
Example	'studies' -> 'studi'	'studies' -> 'study'

6.2 Practical Implementation

WordNetLemmatizer from NLTK is used below. Providing the correct Part-of-Speech (POS) tag noticeably improves the result, so the example also shows lemmatization with and without POS.

```
from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()

words = ['studies', 'studying', 'better', 'playing', 'rocks', 'went']

print(f"{'Word':<12}{'Without POS':<15}{'With POS (verb)':<15}")
for w in words:
    without_pos = lemmatizer.lemmatize(w)
    with_pos = lemmatizer.lemmatize(w, pos='v')
    print(f"{w:<12}{without_pos:<15}{with_pos:<15}")

# 'better' as an adjective needs pos='a'
print("\n'better' lemmatized as adjective:", lemmatizer.lemmatize('better', pos='a'))
```

Output:

Word	Without POS	With POS (verb)
studies	study	study
studying	studying	study
better	better	better
playing	playing	play
rocks	rock	rock
went	went	go

'better' lemmatized as adjective: good

Observation: without a POS tag, the lemmatizer defaults to treating the word as a noun, so 'studying' and 'playing' are left unchanged. Supplying the correct POS ('v' for verb, 'a' for adjective) gives the correct lemma.

7. Bag-of-Words (BoW) Model

7.1 Theory

The Bag-of-Words model is one of the simplest ways to represent text numerically so it can be used by machine learning algorithms. It represents each document as a vector of word counts, ignoring grammar and word order — only the frequency of each word in the document matters (hence 'bag' of words, since the words are thrown into a bag without any structure).

Steps to Build a Bag-of-Words Model

1. Collect all documents in the corpus.
2. Preprocess each document (tokenize, remove stopwords, stem/lemmatize).
3. Build a vocabulary of all unique words across the corpus.
4. For each document, create a vector whose length equals the vocabulary size; each position holds the count of that word in the document.

Example

Consider the following two documents:

- Doc 1: "the cat sat on the mat"
- Doc 2: "the dog sat on the log"

Vocabulary: {the, cat, sat, on, mat, dog, log} — 7 unique words. The resulting BoW matrix:

Document	the	cat	sat	on	mat	dog	log
Doc 1	2	1	1	1	1	0	0
Doc 2	2	0	1	1	0	1	1

Limitations of BoW

- Ignores word order and grammar completely ("dog bites man" and "man bites dog" get very similar vectors).
- Produces very high-dimensional, sparse vectors for large vocabularies.
- Treats every word as equally important, even very common words that don't help distinguish documents.

7.2 Practical Implementation

The CountVectorizer class from scikit-learn builds a Bag-of-Words model directly from raw text.

```
from sklearn.feature_extraction.text import CountVectorizer
import pandas as pd

corpus = [
    'the cat sat on the mat',
    'the dog sat on the log',
    'cats and dogs are great pets'
```

```

]

vectorizer = CountVectorizer()
bow_matrix = vectorizer.fit_transform(corpus)

print("Vocabulary:", vectorizer.get_feature_names_out())
print("\nBag-of-Words Matrix:")
df = pd.DataFrame(bow_matrix.toarray(), columns=vectorizer.get_feature_names_out())
print(df)

```

Output:

Vocabulary: ['and' 'are' 'cat' 'cats' 'dog' 'dogs' 'great' 'log' 'mat' 'on' 'pets' 'sat' 'the']

Bag-of-Words Matrix:

	and	are	cat	cats	dog	dogs	great	log	mat	on	pets	sat	the
0	0	0	1	0	0	0	0	0	1	1	0	1	2
1	0	0	0	0	1	0	0	1	0	1	0	1	2
2	1	1	0	1	0	1	1	0	0	0	1	0	0

8. TF-IDF (Term Frequency – Inverse Document Frequency)

8.1 Theory

TF-IDF is a statistical measure that evaluates how important a word is to a document within a collection (corpus) of documents. It improves upon plain Bag-of-Words by down-weighting words that occur frequently across many documents (and are therefore less informative) and up-weighting words that are frequent in a specific document but rare across the corpus (and are therefore more distinctive).

Formula

TF-IDF is the product of two components:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

- Term Frequency: $\text{TF}(t, d) = (\text{Number of times term } t \text{ appears in document } d) / (\text{Total number of terms in document } d)$
- Inverse Document Frequency: $\text{IDF}(t) = \log(N / (1 + \text{df}(t)))$, where N = total number of documents in the corpus, and $\text{df}(t)$ = number of documents containing term t

The '+1' in the denominator (smoothing) prevents division by zero when a term is absent from all documents. A high TF-IDF score means the word occurs often in this document but rarely in the rest of the corpus — making it a good keyword/distinguishing feature for that document.

BoW vs TF-IDF

Aspect	Bag-of-Words	TF-IDF
What it captures	Raw word frequency/count	Weighted importance of a word
Common words	Given high weight (bad)	Down-weighted automatically
Rare, distinctive words	Treated same as common words	Given higher importance
Typical use	Simple counting tasks	Search engines, text classification, keyword extraction

8.2 Practical Implementation

The `TfidfVectorizer` class from scikit-learn computes the TF-IDF matrix directly from raw text.

```
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

corpus = [
    'the cat sat on the mat',
    'the dog sat on the log',
    'cats and dogs are great pets'
]

tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(corpus)
```

```
df = pd.DataFrame(
    tfidf_matrix.toarray(),
    columns=tfidf_vectorizer.get_feature_names_out()
)
print("TF-IDF Matrix (rounded to 3 decimals):")
print(df.round(3))
```

Output:

TF-IDF Matrix (rounded to 3 decimals):

	and	are	cat	cats	dog	dogs	great	log	mat	on
0	0.000	0.000	0.373	0.000	0.000	0.000	0.000	0.000	0.373	0.283
1	0.000	0.000	0.000	0.000	0.373	0.000	0.000	0.373	0.000	0.283
2	0.354	0.354	0.000	0.354	0.000	0.354	0.354	0.000	0.000	0.000

	pets	sat	the
0	0.000	0.283	0.567
1	0.000	0.283	0.567
2	0.354	0.000	0.000

Observation: the word 'the' has the highest weight in Doc 1 and Doc 2 (because it repeats within the document) but scores 0 in Doc 3 where it never appears; words like 'cats', 'dogs', 'pets' that are unique to Doc 3 receive noticeably higher, more discriminating weights than common words.

9. Complete End-to-End Pipeline

The program below combines every stage covered in this file — tokenization, stopwords removal, lemmatization, Bag-of-Words, and TF-IDF — into a single, reusable preprocessing pipeline applied to a small corpus.

```
import re
import nltk
import pandas as pd
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    text = text.lower()
    text = re.sub(r'^a-z\s', '', text)          # remove punctuation/numbers
    tokens = word_tokenize(text)                # tokenize
    tokens = [t for t in tokens if t not in stop_words] # remove stopwords
    tokens = [lemmatizer.lemmatize(t) for t in tokens] # lemmatize
    return ' '.join(tokens)

corpus = [
    'The cats are running and playing in the garden.',
    'Dogs were barking loudly at the running cats.',
    'Cats and dogs are popular household pets.'
]

cleaned_corpus = [preprocess(doc) for doc in corpus]
print("Cleaned Corpus:")
for c in cleaned_corpus:
    print(' -', c)

# Bag of Words on cleaned text
cv = CountVectorizer()
bow = cv.fit_transform(cleaned_corpus)
print("\nBoW Vocabulary:", cv.get_feature_names_out())

# TF-IDF on cleaned text
tfidf = TfidfVectorizer()
tfidf_matrix = tfidf.fit_transform(cleaned_corpus)
print("\nTF-IDF Matrix:")
print(pd.DataFrame(tfidf_matrix.toarray(),
    columns=tfidf.get_feature_names_out()).round(2))
```

Output:

Cleaned Corpus:

- cat running playing garden
- dog barking loudly running cat
- cat dog popular household pet

Bow Vocabulary: ['barking' 'cat' 'dog' 'garden' 'household' 'loudly'
'playing' 'popular' 'pet' 'running']

TF-IDF Matrix:

	barking	cat	dog	garden	household	loudly	playing	popular	pet
0	0.00	0.35	0.00	0.60	0.00	0.00	0.60	0.00	0.00
1	0.53	0.31	0.40	0.00	0.00	0.53	0.00	0.00	0.00
2	0.00	0.35	0.35	0.00	0.60	0.00	0.00	0.60	0.60